OPERATIONS WORKBOOK

Timed Coding and System Design Rehearsal Workbook

Two controlled, interview-shaped practice blocks with protected decision clocks, evidence rubrics, worksheets, and debriefs.

Use time pressure to test decision order while protecting clarification, invariants, testing, trade-offs, and failure handling.

VERSION 1.0

FORMAT GUIDE + WORKSHEET

STATUS LEARNER READY

OPERATIONS / TIMED-TECHNICAL-REHEARSAL

REHEARSE THE DECISION ORDER

Use the clock without racing

A timed rehearsal tests whether clarification, reasoning, testing, and trade-offs still appear when the interview window is fixed.

CORE IDEA

The timer measures decision order, not intelligence or readiness. Protect the steps that make an answer trustworthy; compress detail before silently skipping the contract, evidence, testing, or failure handling.

01
Set
Choose one coding prompt and one design prompt, then lock time, tools, and assistance.

02
Run
Speak decisions aloud and mark the moments where the answer changes direction.

03
Capture
Save the prompt, artifacts, score, quote, timestamp, and first missed decision.

04
Correct
Choose one behavior to test next; do not turn the debrief into a new syllabus.

SELF-REVIEW
Solo mode
Record audio or screen only when permitted. Use visible timestamps and score from artifacts immediately after the run.

PAIRED REVIEW
Observer mode
Ask the observer to note quotes and decision moments. Neutral prompts may clarify; they must not rescue the solution.

The 80-minute practice block

- [ ] Coding rehearsal: 30-minute attempt plus 10-minute evidence capture.
- [ ] System-design rehearsal: 35-minute attempt plus 5-minute evidence capture.
- [ ] Use the stated clock, the task-specific rubric, one 0 / 1 / 2 evidence scale, and the same assistance policy for each run.
- [ ] Save the required artifacts before choosing a correction.

PREPARE THE CONDITIONS

Select prompts that expose decisions

Use an active Library mock or a prompt you have not recently solved. The goal is retrieval and explanation, not recognition from memory.

Block	Prompt choice	Target dimension	Locked conditions
Coding	One sliding-window, graph, or dynamic-programming prompt	The decision you want the clock to expose	30 min; editor; language; no solution lookup
Design	One URL, notification, storage, or scheduling prompt	Requirements, scale, trade-off, or failure handling	35 min; blank canvas; calculator policy stated
Recording	Audio, screen, observer notes, or written timestamps	Exact quote and decision moment	Permission confirmed; no confidential prompt capture
Evidence	Code and tests; diagram and decision ledger	Observable result, not a feeling	Scored immediately with the same rubric

Setup record

Field	Coding block	System-design block
Prompt / source		
Start time / end time		
Tools / assistance		
Completion evidence		

PROMPT RULE

Use the Library mock as a prompt source, not as an answer script. If you remember the solution path, choose another prompt and record why.

CODING BLOCK

Protect the contract, invariant, and tests

The 30-minute run keeps the roadmap task interview-shaped. The following 10 minutes preserve evidence before memory rewrites the attempt.

Clock	Decision to make	What the learner says or saves
0-5 min	Clarify contract, constraints, and examples	Inputs, outputs, edge rules, one normal example, one boundary example
5-10 min	Choose approach and state invariant	Pattern cue, state, invariant, rejected alternative, expected complexity
10-25 min	Implement with audible checkpoints	What each block preserves; any changed assumption; uncertainty named
25-30 min	Test and close	Empty/minimum, boundary or duplicate, main transition, time and space complexity
30-40 min	Capture evidence	Artifacts, scores, quote/timestamp, first missed edge, one correction

If you are stuck

01
Restate
Say the contract and the exact uncertainty.

02
Shrink
Use the smallest example that forces the main state change.

03
Baseline
Name a correct simple approach before optimizing.

04
Resume
Choose the next observable step and state its cost.

TIME-PRESSURE RULE

Do not hide. A visible correct baseline plus a named optimization gap is stronger evidence than silent guessing or an unexplained code jump.

CODING WORKSHEET

Save the reasoning that produced the code

Complete the left side during clarification and planning. Complete the evidence rows immediately after the timer ends.

Artifact	Your record
Contract and constraints	
Normal + boundary example	
Pattern cue / approach	
State and invariant	
Rejected alternative + reason	
Time / space complexity	
First missed edge + timestamp	

Score	Evidence standard
0 - absent	The behavior never appears or appears only after the run.
1 - prompted / late	The behavior appears after a rescue, late realization, or unexplained change.
2 - independent	The behavior appears at the correct decision point and is supported by an artifact.

CODING EVIDENCE

Score moments, not confidence

Use one evidence scale across dimensions. A total is useful only when each mark points to something the learner said, wrote, implemented, or tested.

Dimension	0 / 1 / 2	Quote, timestamp, code, or test evidence
Clarification / contract		
Approach / decision order		
State / invariant		
Correctness / recovery		
Complexity / alternatives		
Testing / communication		

Worked evidence example

Observed moment	Cause hypothesis	Smallest correction
Invariant stated at minute 18 after implementation drifted	Planning named the pattern but not what the window must preserve	State the invariant before the first loop and re-check it at each pointer move
Final duplicate boundary test was missed	Testing started without a written edge list	Write four test labels during clarification; run them in the last five minutes

DO NOT COPY THE EXAMPLE

Use it to separate symptom, cause, and correction. Your correction must attach to your own evidence and decision cue.

SYSTEM-DESIGN BLOCK

Make assumptions reversible

A strong timed design answer connects requirements and scale to components, then closes with bottlenecks, recovery, and one deliberate trade-off.

Clock	Decision to make	What the learner says or saves
0-5 min	Requirements and non-goals	Users, core actions, latency/durability needs, boundaries, explicit exclusions
5-9 min	Order-of-magnitude scale	Traffic, storage, peaks, retention, and which estimate changes the design
9-17 min	API, data, and main flow	Contract, source of truth, write/read path, key entities, access pattern
17-25 min	Architecture and bottleneck	Components, ownership boundaries, first saturation or contention point
25-31 min	Failure and recovery	Failure mode, detection signal, containment, retry/idempotency or fallback
31-35 min	Trade-off and operations close	Alternative rejected, consequence accepted, observability, next question
35-40 min	Capture evidence	Diagram, three assumptions, decision ledger, score, quote, correction

COMPRESSION RULE

When time is short, reduce breadth and say what is deferred. Never silently skip requirements, a source of truth, the primary bottleneck, or failure handling.

DESIGN WORKSHEET

Show what each assumption caused

The three-row decision ledger is the minimum artifact for the roadmap task. Each row must make a design choice revisitable.

Assumption	Decision caused	Cost / trade-off	What would change it

Design evidence	Your record
Source of truth + data flow	
Primary bottleneck + limit	
Failure / recovery / metric	
Trade-off to revisit	

DECISION TEST

If an assumption changes, point to the component, data path, or operating policy that changes with it. If nothing changes, the assumption may not be decision-relevant.

DESIGN EVIDENCE

Score the answer and test one pivot

Use the same 0 / 1 / 2 evidence scale. Then introduce one nearby requirement change and observe whether the learner updates only the affected decisions.

Dimension	0 / 1 / 2	Quote, timestamp, diagram, or decision evidence
Requirements / non-goals		
Scale / capacity		
Decision order / data flow		
Architecture / bottleneck		
Trade-off clarity		
Failure / recovery / metrics		
Communication / recovery		

Worked pivot

Requirement change	What changes	What stays stable
Delivery may be delayed, but duplicate actions are unacceptable	Idempotency key, deduplication store, retry policy, and success metric become explicit	The product contract, source of truth, and unrelated read path remain unchanged

NEUTRAL OBSERVER PROMPTS

Use: What assumption drives that? What fails first? Which decision changes if the requirement changes? Avoid suggesting the component or answer.

FINISH BOTH TASKS

Leave with artifacts and one next correction

The rehearsal is complete when both runs are saved, evidence is scored, and the next test is smaller than the practice block you just finished.

Roadmap task	Required saved artifact	Completion proof
Timed coding rehearsal	Prompt, approach, invariant, complexity, test list, first missed edge	One quote/timestamp and a rubric score tied to code or tests
Timed system-design rehearsal	Prompt, three assumptions and caused decisions, diagram, trade-off to revisit	One quote/timestamp and a rubric score tied to the decision ledger

Common mistakes

+ Starting implementation or architecture before clarifying the contract.
+ Optimizing for speed while dropping tests, failure handling, or a clear close.
+ Using a familiar prompt whose solution is already memorized.
+ Scoring from confidence instead of quotes, artifacts, and decision moments.
+ Choosing several corrections and losing evidence about what worked.

YOUR NEXT ACTION

Choose one lowest evidence dimension, write one correction sentence, and schedule one comparable rehearsal. Keep the prompt shape and rubric stable enough to measure change.

Completion check

- [ ] Both prompts and conditions are recorded.
- [ ] Both clocks preserve the required decision stages.
- [ ] Coding and design artifacts are saved.
- [ ] Scores cite observable evidence.
- [ ] Only one correction is active for the next test.